

Introduction.

- Ballpark how close parts of your model are to their theoretical optimum
- make informed choices about different parallelism schemes at different scales.
- Estimate the cost and time required to train the large transformer model.
- Design algorithms that take advantage of specific hardware affordances.
- Design hardware driven by an explicit understanding of what limits current algorithm performance

* Questions to answer

- How long should a matrix multiply of a certain task take? At what point is it compute bound vs memory bound?
- How are TPUs wired together to form training clusters? How much bandwidth does each part of the system have?
- How long does it take to gather, scatter or re-distribute arrays across multiple TPUs?
- How do we efficiently multiply matrices that are distributed differently across devices?

1. Intro to offline.

- Once we run algorithm on hardware, we are bound by three things:
 - how fast our computer can do math
 - the bandwidth available for moving data
 - total memory available to store data.

• Computation:-

$$T_{\text{math}} = \frac{\text{Computation FLOPs}}{\text{Accelerator FLOPs/second}}$$

$$\cdot H100 \rightarrow 9.89e^{14} \text{ bf16 Flops/s.}$$

Communication within the chip

Communication b/w chips

$$T_{\text{communication}} = \frac{\text{Communication Bytes}}{\text{Network memory b/w Bytes/s}}$$

Typically, computation within a single chip can be overlapped with communication within a chip & b/w chips. This means we can lower bound training & inference time by using the max of computation & communication time.

We can also upper-bound with their sum

$$T_{\text{lower}} = \max(T_{\text{math}}, T_{\text{comm}})$$

$$T_{\text{upper}} = T_{\text{math}} + T_{\text{comm}}$$

When $T_{\text{math}} > T_{\text{comm}}$, meaning $\geq$ compute bound & when $T_{\text{comm}} > T_{\text{math}}$, we are memory bound.

* Arithmetic intensity $\rightarrow$ ratio of total FLOPs it performs to the number of bytes it needs to communicate

$$\text{Arithmetic intensity} = \frac{\text{Computation FLOPs}}{\text{Communication Bytes}}$$

Example: 1

$$x \in \mathbb{R}^N, y \in \mathbb{R}^N, z \in \mathbb{R}^1$$

$$\text{dot}(x, y) = x \odot y = z$$

$$\text{total bytes loaded} = (N+N) \times 2 = 4N$$

$$\text{computation (FLOPs)} = N+N-1 = 2N-1 = \underline{2N}$$

$$\text{output} = 2 \text{ bytes} = 2 \text{ bytes}$$

$$\text{Intensity} = \frac{2N-1}{4N+2} \approx \frac{1}{2} \quad (N \rightarrow \infty)$$

how do you visualize the tradeoff b/w memory & compute

classmate

Date
Page

* Visualizing routines:

- How does FLOPs/second correlate with arithmetic intensity?

F = total FLOPs

T = time taken to move

$$\text{FLOPs/second} = \frac{F}{T}$$

$$T \approx \max(T_{\text{comput}}, T_{\text{comm}})$$

$$= \max\left(\frac{F}{\text{max FLOPs}}, \frac{B}{\text{BW}}\right)$$

Asymmetric

$$\text{intensity} = \frac{F}{B} \quad T = \max\left(\frac{F}{\text{peak FLOPs}}, \frac{F}{\text{AI} \cdot \text{BW}}\right)$$

$$\text{FLOPs/second} = \frac{1}{\max\left(\frac{1}{\text{peak FLOPs}}, \frac{1}{\text{AI} \cdot \text{BW}}\right)} = \min(\text{peak FLOPs}, \text{AI} \cdot \text{BW})$$

$$\text{FLOPs} = \frac{F}{T} = \frac{1}{\max\left(\frac{1}{\text{peak FLOPs}}, \frac{1}{\text{AI} \cdot \text{BW}}\right)} = \min(\text{peak FLOPs}, \text{AI} \cdot \text{BW})$$

What question can I get an answer to by following this plot?

Given my algorithm's AI, what fraction of peak can I actually get on this hardware, and is the fix more bandwidth, more compute, or restructuring the algo to raise intensity.

1.1.2 matrix multiplication

$$X * Y = Z, \quad X \in \mathbb{R}^{B \times D}, \quad Y \in \mathbb{R}^{D \times F} \quad \left. \vphantom{\begin{matrix} X \\ Y \\ Z \end{matrix}} \right\} \text{BF16}$$

matrix multiplication

$$\text{load} = 2 \times (BD + DF) + 2 \times BF$$

reading writing

$$\text{FLOPs} = B(D + D - 1)F = B(2D - 1)F$$

$$A.F = \frac{2BDF}{2(BD+DF+BF)}$$

$$\frac{BDF}{BD+DF+BF}$$

Now, if B is small compared to D & F

$$A.F \approx B$$

$$I_m(\text{matmul}) > I(\text{cub.})$$

$$B > 240$$

1.1.3 network bottleneck

split on two chips

$$X \in \mathbb{R}^{B \times D}$$

$$Y \in \mathbb{R}^{D \times F}$$

$$Z \in \mathbb{R}^{B \times F}$$

Split on p , & then aggregate final results

$$T_{\text{computation}} = \frac{\text{FLOPs}}{(\text{per GPU})} = \frac{2BD/2 \cdot F}{\text{peak FLOPs}} = \frac{BDF}{\text{peak FLOPs}}$$

$$T_{\text{comm.}} = \frac{2 \times \left(\frac{BD}{2} + \frac{DF}{2} \right) + 2 \times BF}{(\text{per GPU}) \cdot \text{peak BW}}$$

$$= \frac{BD + DF + 2BF}{\text{peak BW}}$$

$$T_{\text{comm. sharing}} = \frac{2(2BF) + BF}{\text{peak BW}}$$

$$T_{\text{computat. sharing}} = \frac{BF}{\text{peak FLOPs}}$$

1.2 Worked problems:

1.1

$$X \in \mathbb{R}^{B \times D}, Y \in \mathbb{R}^{D \times F} \text{ \& \& } Z \in \mathbb{R}^{B \times F} \text{ int } B(16 \text{yH})$$

① Bytes load = $BD + DF$

② written back = BF

③ total ops = $BF(2D-1)$

④ $A.F = \frac{BF(2D-1)}{BD+DF+BF}$

$$= \frac{2BFD}{BD+DF+BF}$$

$$= \frac{2BFD}{BD+DF+BF}$$

HBm b/w is $8.1e^{11}$ bytes/second, peak FLOP $\approx 3.94e^{14}$.

$$T_{math} = \frac{2BPD}{3.9e^{14}} T_{comm.} = \frac{BD + DF + BF}{3.9e^{14} \cdot 8.1e^{11}}$$

compute bound. $2B \approx 8.1e^{11}$

$$\frac{2B \approx 3.9e^{14}}{8.1e^{11}}$$

Upper & lower bound.

$$Runtime = T_{math} + T_{compute}$$

$$\text{lower bound} = \max \left(\frac{2BPD}{3.9e^{14}}, \frac{BD + DF}{8.1e^{11}} \right) \quad (\text{assume } B \text{ is small})$$

$$\text{Upper bound} = 3.9 T_{math} + T_{compute}$$

What is the difference between biton and torch compile?

What is PTX?

A. Diff between biton and torch compile.

• torch compile generates biton & biton usually compiles down to PTX.

A. PTX $\rightarrow$ parallel thread execution is NVIDIA's virtual instruction set. It's an assembly looking human-readable language with registers, loads & stores, and thread & predicate semantics.

Ex-1.2

weights $\rightarrow$ in low precision

activation $\rightarrow$ in higher precision

Q. What batch size do we become compute bound.

$$T_{computation} \geq T_{communication}$$

$$\text{bfloat16}[B, D] * \text{int8}[D, P] \rightarrow \text{bfloat16}[B, P]$$

$$T_{\text{computation}} = \frac{2BDF}{2BD + DF + 2BF}$$

$$T_{\text{computation}} = \frac{2BDF}{2BD + DF + 2BF}$$

$$T_{\text{communication}} = \frac{\text{peak flops}}{2BD + DF + 2BF}$$

$$T_{\text{computation}} \gg T_{\text{communication}}$$

$$\frac{2BDF}{\text{peak flops}} \gg \frac{2BD + DF + 2BF}{\text{bw}}$$

$$B \gg \frac{1}{2} \left(\frac{\text{peak flops}}{\text{bw}} \right)$$

Exercise - 1.3 Make a roughline plot of peak flops vs B for several values of D & F

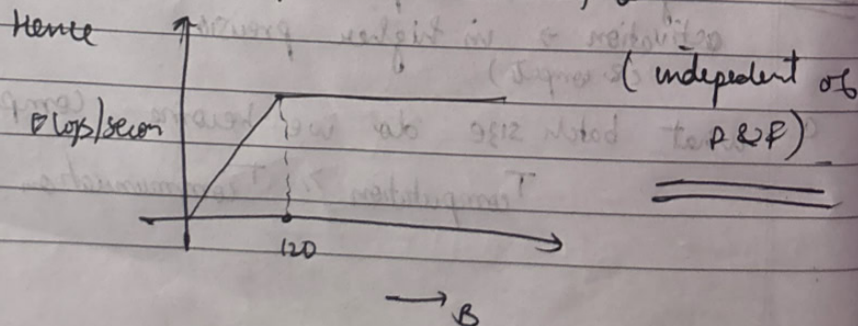
$$\text{Arithmetic I.} = \frac{2BDF}{2BD + DF + 2BF}$$

$$T = \frac{F}{2BD + DF + 2BF}$$

$$\text{① - } \frac{F}{2B} \quad (\text{assume } B \text{ is small})$$

$$\text{Flops/second} = \min(\text{peak flops, } A.S.BW)$$

$$\text{Assumption: } 2B(D+F) \leq DF$$



Ex-1.4. $\text{ints}[B,D] * \text{ints}[B,D,F] \rightarrow \text{ints}[B,F]$,
 where we imagine having a different matrix for
 each batch element. What is the AI of this operation?

→ ~~800~~: Idea is this is a dot product

loading bytes = $BD + BDF$

writing bytes = BF

Matrix multiplication =

• For each batch element, we have

→ $[1,D]$ & $[D,F]$

⇒ $[1,F]$ } → concatenate
 overbake

$$= B \times (2DF)$$

Hence, $AI = \frac{2BDF}{BD + BDF + BF}$, now if $BDF \gg BD + DF$

$AI = 2$ } → compute bound.

$$\text{peak AI} = \frac{3.99e^{14}}{8.1e^{11}} \approx 486$$

1.7. Calculate the batch size at which a matrix multiplication will become compute bound.

~~FLOPs~~
~~B = 16~~

$$2B \gg 2 \times 2000 \times \text{FLOPs} \quad (\text{FLOPs} = 2000 \text{ TFlops})$$

$$B \gg 2000$$

$\frac{1}{2}$ of them some repeated

$$\frac{2BDF}{BD + BDF + BF}$$

$$\text{FLOPs} = 2BDF$$

$$\text{load bytes} = 2(BD + DF + BF)$$

$T_{\text{comput.}}$ > $T_{\text{comm.}}$

$2BDP$

Peak FLOPs

$2BD + 2BF + 2DF$

$3TB/s$

B

1000 TFlops

$\approx$

$\frac{1}{3TB}$

comes from

$\frac{1}{2} (2000 \text{ TFlops})$

$B \approx 333$